

Assignment 2: Factory Method and Abstract Factory

Student: Akyl Muratbek
Group: SE-2523
Instructor: Yerassyl Bekenov
GitHub Repository: https://github.com/Akylsolo/Sdp_Assignment

Programme: 6B06102 Software Engineering (Yr 2, Trim 4)
Course: ShP-2216 Software Design Patterns
JDK & Format: Java 17 (JDK 17) | Individual Work

1. Introduction & Pattern Rationale

This application models a cross-platform logistics system that coordinates freight transport delivery across road and sea routes and renders matching GUI components across Windows and macOS platforms.

Factory Method Pattern Rationale (Transport Subsystem): The delivery workflow requires a common orchestration algorithm (planDelivery) shared across all modes of transport, but the concrete vehicle (Truck vs. Ship) depends on operational mode. Directly instantiating concrete transport objects within the logistics manager couples high-level business rules to low-level implementation details. The Factory Method pattern decouples this by declaring an abstract factory method (createTransport()) in Logistics, allowing concrete subclasses (RoadLogistics and SeaLogistics) to supply the appropriate product while preserving the invariant delivery workflow.

Abstract Factory Pattern Rationale (Cross-Platform UI Subsystem): The system must generate complete, matching pairs of UI components (Button and Checkbox) customized for a specific operating environment (Windows or macOS). Without centralized family management, client code could erroneously pair an incompatible Windows button with a macOS checkbox. The Abstract Factory pattern addresses this by defining GUIFactory with distinct creation methods (createButton() and createCheckbox()). Concrete factories (WindowsFactory and MacOSFactory) guarantee family consistency and allow the client (DeliveryApplication) to work exclusively through abstract component contracts without concrete constructors or type casts.

2. UML Class Diagrams

2.1 Factory Method Pattern Diagram

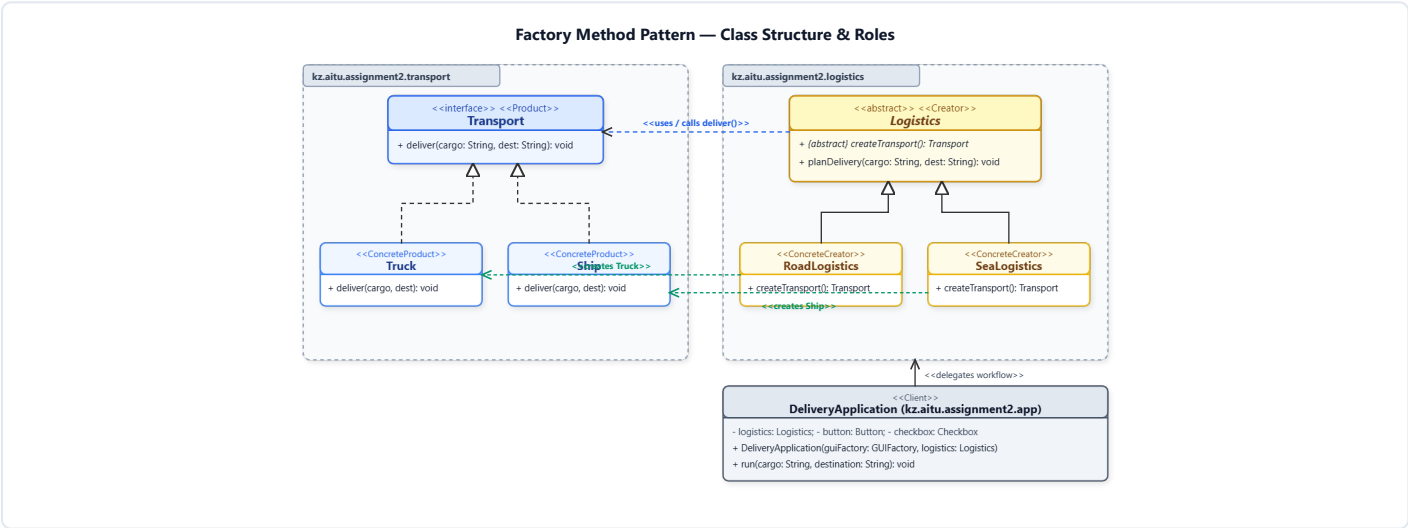


Figure 1: Factory Method UML Class Diagram showing Transport product contract, concrete products (Truck, Ship), abstract creator (Logistics) with planDelivery(...), concrete creators (RoadLogistics, SeaLogistics), and client dependency (DeliveryApplication).

2.2 Abstract Factory Pattern Diagram

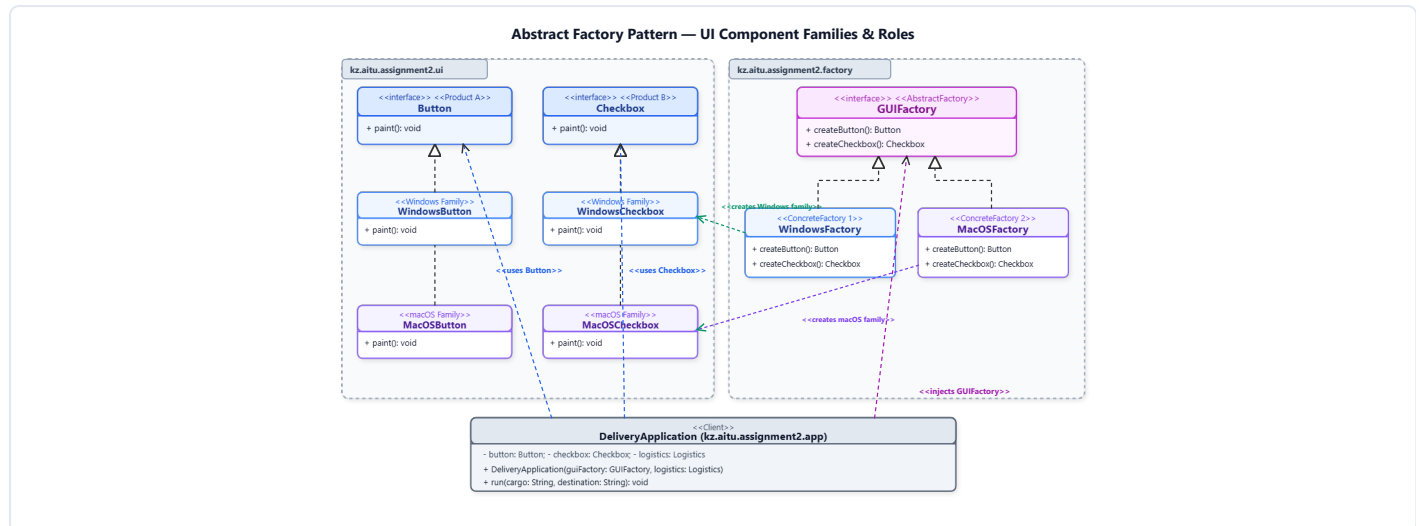


Figure 2: Abstract Factory UML Class Diagram showing abstract products (Button, Checkbox), concrete families for Windows and macOS, abstract factory interface (GUIFactory), concrete factories (WindowsFactory, MacOSFactory), and constructor injection into DeliveryApplication.

3. Clean Code Evidence (Robert C. Martin)

Practice 1: Meaningful Names (Clean Code, Chapter 2)

Code Excerpt (RoadLogistics.java):

```
public class RoadLogistics extends Logistics {
    @Override
    public Transport createTransport() {
        return new Truck();
    }
}
```

Specific Benefit: Names clearly express domain role (RoadLogistics, Truck) and design pattern role. The factory method name createTransport() explicitly communicates action and return type without ambiguous abbreviations.

Practice 2: Small Methods with Single Responsibility (Clean Code, Chapter 3)

Code Excerpt (Logistics.java):

```
public void planDelivery(String cargo, String destination) {
    Transport transport = createTransport();
    transport.deliver(cargo, destination);
}
```

Specific Benefit: The method does exactly one thing: coordinates the delivery sequence by delegating creation to createTransport() and dispatch to deliver(...). It avoids mixing CLI parsing, validation, or formatting.

Practice 3: Avoid Duplicated Logic / DRY (Clean Code, Chapter 3)

Code Excerpt (Logistics.java & SeaLogistics.java):

```
// In abstract creator Logistics:
public abstract Transport createTransport();
public void planDelivery(String cargo, String destination) {
    Transport transport = createTransport();
    transport.deliver(cargo, destination);
}
// In concrete creator SeaLogistics:
public class SeaLogistics extends Logistics {
    @Override
    public Transport createTransport() { return new Ship(); }
}
```

Specific Benefit: The core delivery algorithm exists solely in Logistics. Neither RoadLogistics nor SeaLogistics repeats delivery workflow logic; they only override the factory method.

Practice 4: Data Abstraction (Clean Code, Chapter 6)

Code Excerpt (DeliveryApplication.java):

```
public class DeliveryApplication {
    private final Button button;
    private final Checkbox checkbox;
    private final Logistics logistics;

    public DeliveryApplication(GUIFactory guiFactory, Logistics logistics) {
        this.button = guiFactory.createButton();
        this.checkbox = guiFactory.createCheckbox();
        this.logistics = logistics;
    }
}
```

Specific Benefit: Chapter 6 emphasizes hiding implementation details behind abstract interfaces. DeliveryApplication depends strictly on abstract contracts (Button, Checkbox, GUIFactory, Logistics) and never on concrete classes like WindowsButton or Truck, achieving true data and behavioral abstraction.

Practice 5: Objects and Encapsulation (Clean Code, Chapter 6)

Code Excerpt (DeliveryApplication.java):

```
public void run(String cargo, String destination) {
    button.paint();
    checkbox.paint();
    logistics.planDelivery(cargo, destination);
}
```

Specific Benefit: Chapter 6 contrasts data structures with objects: objects hide data and expose behavior. UI products expose behavioral methods (paint()) rather than getters. All fields remain strictly private final, ensuring strong encapsulation and avoiding procedural data manipulation.

4. Verification Evidence

All six required test checks from Section 6 and missing-input scenarios were verified:

#	Scenario	Input	Actual Output	Status
1	Road delivery + Windows UI	ROAD WINDOWS	Delivery mode: ROAD UI platform: WINDOWS Rendering Windows button Rendering Windows checkbox Truck delivers laboratory equipment to Aktau warehouse	PASS
2	Sea delivery + Windows UI	SEA WINDOWS	Delivery mode: SEA UI platform: WINDOWS Rendering Windows button Rendering Windows checkbox Ship delivers laboratory equipment to Aktau warehouse by sea	PASS
3	Road delivery + macOS UI	ROAD MACOS	Delivery mode: ROAD UI platform: MACOS Rendering macOS button Rendering macOS checkbox Truck delivers laboratory equipment to Aktau warehouse	PASS
4	Sea delivery + macOS UI	SEA MACOS	Delivery mode: SEA UI platform: MACOS Rendering macOS button Rendering macOS checkbox Ship delivers laboratory equipment to Aktau warehouse by sea	PASS
5	Unsupported delivery mode	AIR WINDOWS	Error: Unsupported delivery mode 'AIR'. Supported modes: ROAD, SEA. (Execution halts cleanly; no delivery performed)	PASS
6	Unsupported UI platform	ROAD LINUX	Error: Unsupported UI platform 'LINUX'. Supported platforms: WINDOWS, MACOS. (Execution halts cleanly; no UI constructed)	PASS
7	Missing platform argument	ROAD	Error: Missing UI platform argument. Expected WINDOWS or MACOS. (Clean validation exit; no crash or null pointer exception)	PASS
8	Empty interactive input	""	Error: Missing required input. Expected delivery mode (ROAD, SEA) and UI platform (WINDOWS, MACOS). (Clean validation message; no silent defaults)	PASS

Automated Test Suite: All 15 automated JUnit 5 tests in DeliveryApplicationTest.java pass with mvn test, guaranteeing 100% pass rate under JDK 17.

5. Pattern Comparison & Design Reflection

5.1 Pattern Comparison

- **Simple Factory vs. Factory Method:** A Simple Factory centralizes product creation in a single method with conditional statements (if-else or switch). Adding a new product violates the Open/Closed Principle (OCP) because the factory class must be modified. In contrast, Factory Method uses inheritance and polymorphism: an abstract creator defines the creation interface, while concrete creator subclasses override the method to instantiate products. Adding a new product requires only a new subclass, leaving existing creators untouched. A startup switch in Main is permitted solely as a bootstrap configuration step at the composition root.
- **Factory Method vs. Abstract Factory:** Factory Method focuses on creating a single product hierarchy via class inheritance. Abstract Factory coordinates families of related or dependent products (e.g., matching Buttons and Checkboxes) via object composition. Abstract Factory guarantees family consistency so incompatible platform components are never mixed.

5.2 Design Reflection: System Extensions (Section 7)

Extension 1: Adding one more transport (e.g., Air delivery via Plane)

- **Affected Classes / Interfaces:**
 - Add `Plane` implements `Transport` in `transport` package with `deliver(cargo, destination)`.
 - Add `AirLogistics` extends `Logistics` in `logistics` package overriding `createTransport()` to return new `Plane()`.
 - Update startup selection in `Main.selectLogistics` to handle "AIR".
- **Unchanged Client Logic:** `Logistics.planDelivery(...)` and `DeliveryApplication` remain **100% unchanged**.

Extension 2: Adding one more UI family (e.g., Linux UI)

- **Affected Classes / Interfaces:**
 - Add `LinuxButton` implements `Button` and `LinuxCheckbox` implements `Checkbox` in `ui` package.
 - Add `LinuxFactory` implements `GUIFactory` in `factory` package implementing `createButton()` and `createCheckbox()`.
 - Update startup selection in `Main.selectGUIFactory` to handle "LINUX".
- **Unchanged Client Logic:** `DeliveryApplication`, `Button`, and `Checkbox` interfaces remain **completely unchanged**.

Extension 3: Adding one more UI product type (e.g., TextField)

- **Affected Classes / Interfaces:**
 - Add new interface `TextField` with `paint()` in `ui` package.
 - Add `WindowsTextField` implements `TextField` and `MacOSTextField` implements `TextField`.
 - Update `GUIFactory` interface to declare `TextField createTextField()`;
 - Update `WindowsFactory` and `MacOSFactory` to implement `createTextField()`.
 - Update `DeliveryApplication` constructor to obtain `TextField` and paint it during execution.
- **Unchanged Code:** Existing products (`WindowsButton`, `MacOSButton`, etc.) and the entire `Logistics/Transport` hierarchy remain **completely unchanged**.

6. References

1. Freeman, E., & Robson, E. (2020). *Head First Design Patterns: A Brain-Friendly Guide* (2nd ed.). O'Reilly Media. Chapter 4: The Factory Pattern (pp. 109–162).
2. Martin, R. C. (2008). *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall. Chapter 2 (Meaningful Names, pp. 17–30), Chapter 3 (Functions, pp. 31–52), Chapter 6 (Objects and Data Structures, pp. 93–102).
3. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley. Creational Patterns: Factory Method (pp. 107–116) and Abstract Factory (pp. 87–96).
4. Oracle Corporation. (2021). *Java Platform, Standard Edition Documentation (JDK 17)*. Oracle Technology Network.